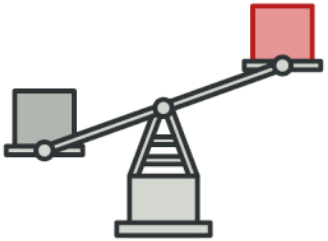
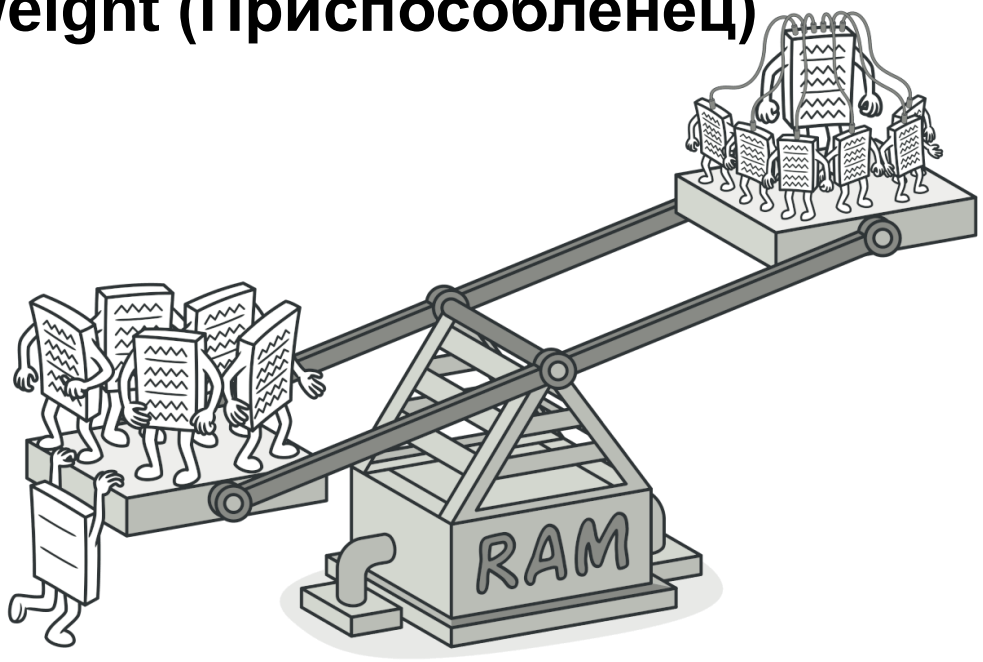
Структурные паттерны. Легковес



Паттерн Легковес
Flyweight (Приспособленец)



Паттерн **Flyweight** – описывает правильное применение техники создания «разделяемых объектов», для получения возможности эффективного использования большого числа объектов.
Экономя памяти достигается благодаря разделению общего состояния, вынесенного в один объект, между множеством объектов.
Легковес позволяет экономить память, кешируя одинаковые данные, используемые в разных объектах.

Проблема

Разделение «*программной сущности уровня выполнения*» на *объект* и *экземпляры*, организовано с целью экономии оперативной памяти.

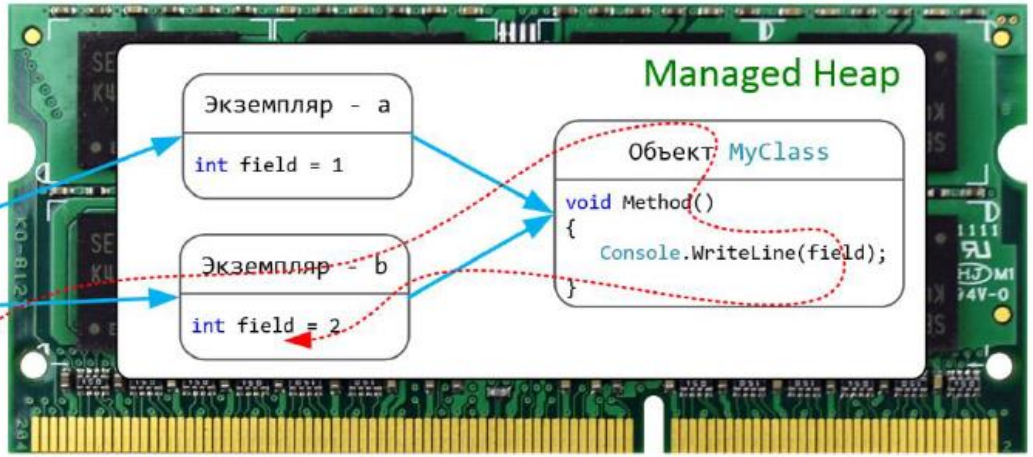
```
class MyClass
{
    public int field;

    public void Method()
    {
        Console.WriteLine(field);
    }
}
```

```
class Program
{
    static void Main()
    {
        MyClass a = new MyClass();
        MyClass b = new MyClass();

        a.field = 1;
        b.field = 2;

        a.Method();
        b.Method();
    }
}
```



у класса может быть только один *объект* и сколько угодно *экземпляров*

Исполняемая сущность

Объект
область динамической памяти, которая содержит в себе *методы и статические поля (разделяемая сущность)*

Экземпляр
область динамической памяти, которая содержит в себе только *нестатические поля (неразделяемая сущность)*

Проблема

«внутреннее состояние» и «внешнее состояние»

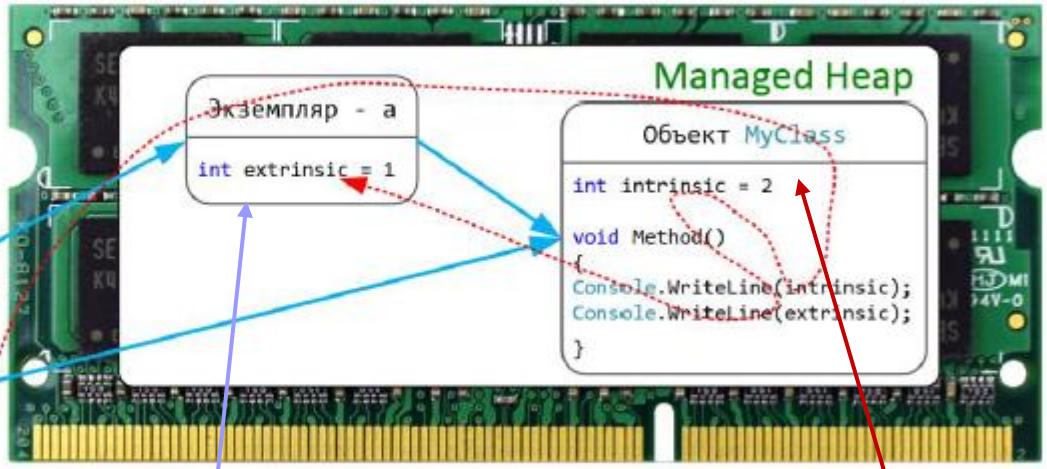
```
class MyClass
{
    public static int intrinsic;
    public int extrinsic;

    public void Method()
    {
    Console.WriteLine(intrinsic);
    Console.WriteLine(extrinsic);
    }
}
```

```
class Program
{
    static void Main()
    {
        MyClass a = new MyClass();
        a.extrinsic = 1;

        MyClass.intrinsic = 2;

        a.Method();
    }
}
```



«*неразделяемый объект*» - «*внешнее состояние*» (изменяемые данные)

«*разделяемый объект*» - «*внутреннее состояние*» (неизменяемые данные)

Проблема

Возможно использовать только ***объект*** без построения ***экземпляра***.

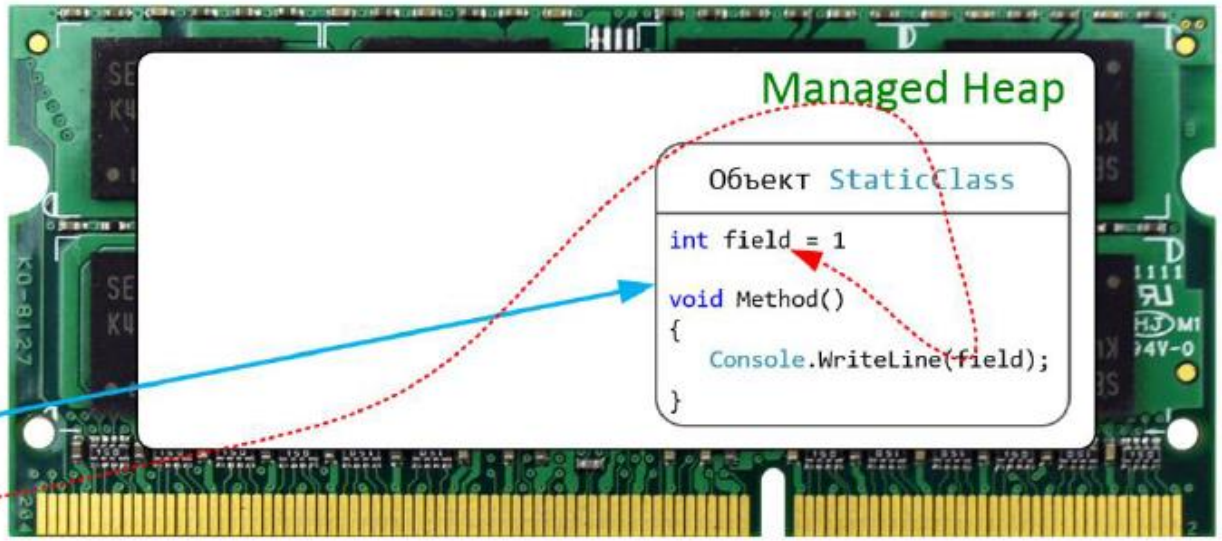
Это возможно при наличии в программе ***статического класса*** или ***статических членов*** нестатического класса

Построение только одного **экземпляра «*неразделяемого*», без объекта - невозможно**

```
static class StaticClass
{
    public static int field;

    public static void Method()
    {
        Console.WriteLine(field);
    }
}
```

```
class Program
{
    static void Main()
    {
        StaticClass.field = 1;
        StaticClass.Method();
    }
}
```



Решение

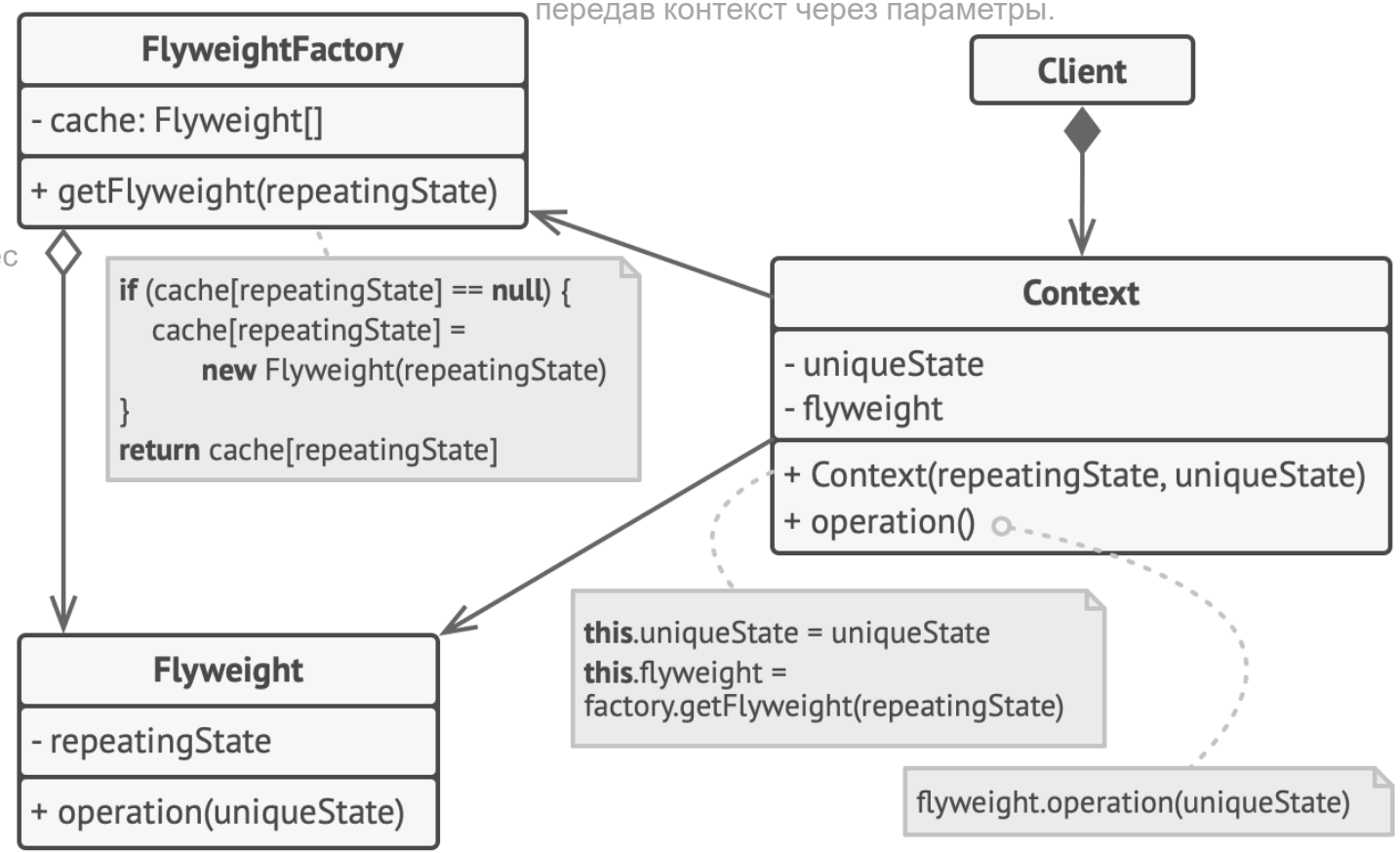
Паттерн **Легковес** – как дополнение разделения «*программных сущностей уровня выполнения*» на объекты и экземпляры

- **Легковес** целесообразно применять в программе, имеющей большое количество одинаковых объектов. Их количество должно быть критично, с точки зрения возможности размещения в доступной памяти.
- Паттерн разделяет данные этих объектов на две части — ***легковесы*** и ***контексты***.
- Всё ***внутреннее*** состояние легковес должен получать через параметры конструктора. Он не должен иметь сеттеров и публичных полей.
- Для удобства работы с легковесами и контекстами можно создать фабричный метод, принимающий в параметрах всё внутреннее состояние нужного объекта.
- Фабричный метода позволит обнаружить уже созданные легковесы с таким же внутренним состоянием, что и требуемое. Если легковес существует - его можно повторно использовать. Если нет - создаётся новый.

Структура паттерна

[4]**Клиент** вычисляет или хранит *контекст - внешнее состояние* легковесов. Для клиента легковесы выглядят как шаблонные объекты, которые можно настроить во время использования, передав контекст через параметры.

[5]**Фабрика легковесов** управляет созданием и повторным использованием легковесов. Фабрика получает запросы, в которых указано желаемое состояние легковеса. Если легковес с таким состоянием уже создан, фабрика сразу его возвращает, а если нет — создаёт новый объект.

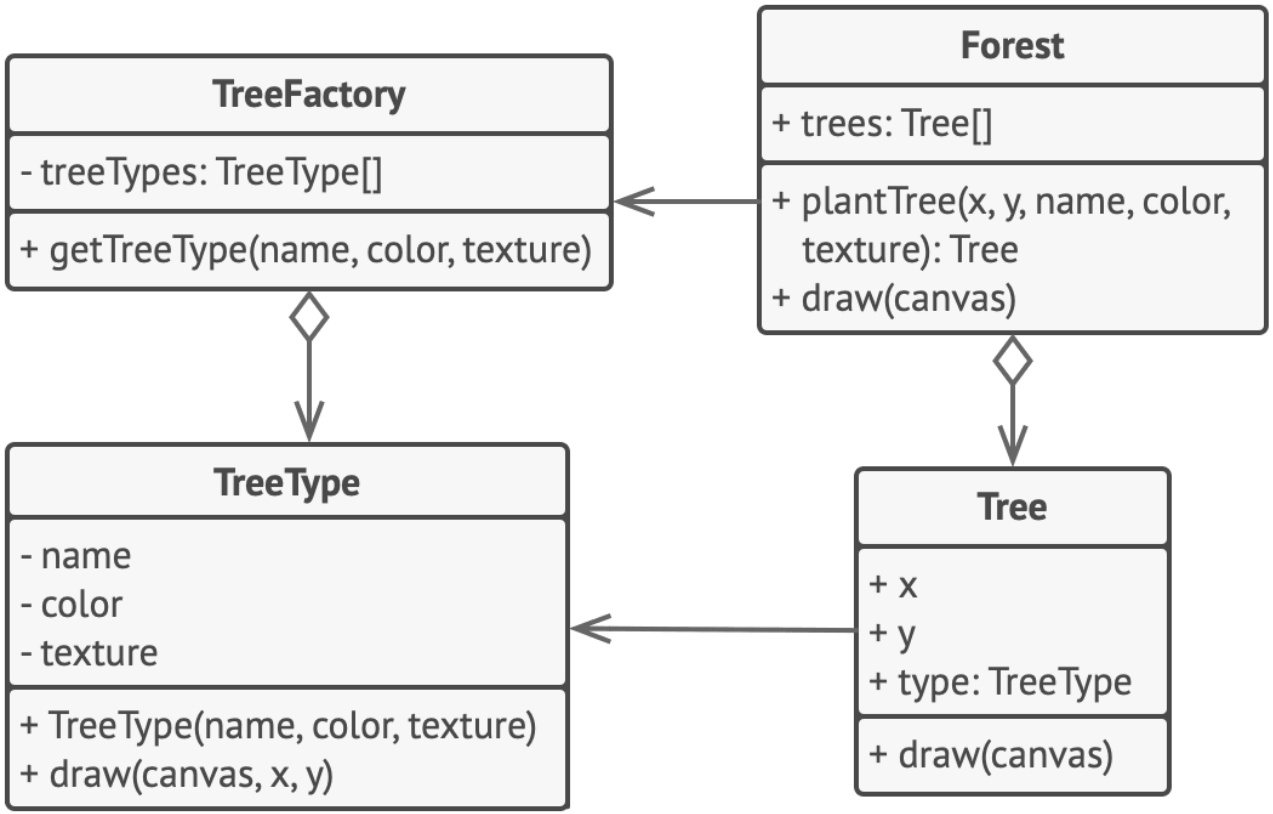


[1]**Легковес** содержит состояние, которое повторялось во множестве первоначальных объектов. Один и тот же легковес можно использовать в связке со множеством контекстов. Состояние, которое хранится здесь, называется *внутренним*, а то, которое он получает извне — *внешним*.

[2]**Контекст** содержит «внешнюю» часть состояния, уникальную для каждого объекта. Контекст связан с одним из объектов-легковесов, хранящих оставшееся состояние.

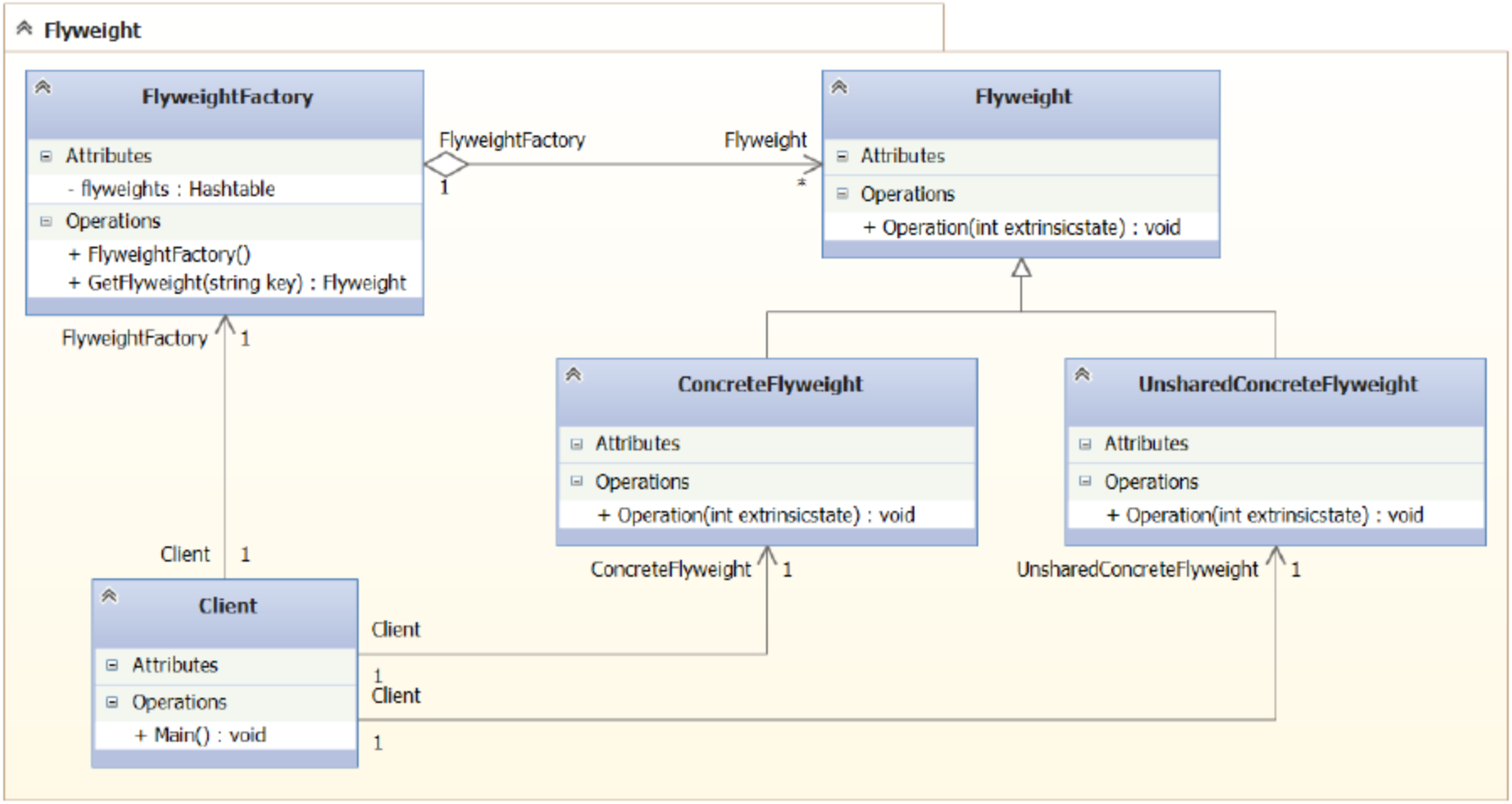
[3]Поведение оригинального объекта чаще всего оставляют в *Легковесе*, передавая значения *контекста* через параметры методов. Тем не менее, поведение можно поместить и в контекст, используя легковес как объект данных.

Лес



- *Легковес* выделяет повторяющуюся часть состояния из основного класса **Tree** и помещает его в дополнительный класс **TreeType**.
- Вместо хранения повторяющихся данных во всех объектах, отдельные деревья будут ссылаться на несколько общих объектов, хранящих эти данные.
- Клиент работает с деревьями через фабрику деревьев, которая скрывает от него сложность кеширования общих данных деревьев.

Структура паттерна UML



Структура паттерна на языке C#

```
class FlyweightFactory
{
    Hashtable pool = new Hashtable
    {
        {"1", new ConcreteFlyweight()},
        {"2", new ConcreteFlyweight()},
        {"3", new ConcreteFlyweight()}
    };

    public Flyweight GetFlyweight(string key)
    {
        if (!pool.ContainsKey(key))
            pool.Add(key, new ConcreteFlyweight());

        return pool[key] as Flyweight;
    }
}
```

`Flyweight` – **Легковес**:
Предоставляет интерфейс, при помощи которого *разделяемые* объекты (*легковесы*) могут подключать *внешнее* состояние и воздействовать на него.

```
abstract class Flyweight
{
    public abstract void Operation(int extrinsicState);
}
```

```
class ConcreteFlyweight : Flyweight
{
    int intrinsicState;

    public override void Operation(int extrinsicState)
    {
        intrinsicState = extrinsicState;
    }
}
```

```
class UnsharedConcreteFlyweight : Flyweight
{
    int allState;

    public override void Operation(int extrinsicState)
    {
        allState = extrinsicState;
    }
}
```

`ConcreteFlyweight` – **Разделяемый конкретный легковес**:
Класс разделяемого объекта. Реализует интерфейс класса `Flyweight` и добавляет при необходимости внутреннее состояние.

```
class Program
{
    static void Main()
    {
        int extrinsicState = 0;

        Flyweight flyweight = null;
        FlyweightFactory factory = new FlyweightFactory();

        flyweight = factory.GetFlyweight("1");
        flyweight.Operation(extrinsicState);

        flyweight = factory.GetFlyweight("10");
        flyweight.Operation(extrinsicState);

        flyweight = new UnsharedConcreteFlyweight();
        flyweight.Operation(extrinsicState);
    }
}
```

`Client` – **Клиент**:
Работает с *разделяемыми* и *неразделяемыми* объектами. Формирует и может хранить *внешнее* состояние *разделяемых* объектов.

UnsharedConcreteFlyweight – **Неразделяемый конкретный легковес**:
Легковесами принято называть *разделяемые* объекты. Неразделяемый легковес – «*неразделяемо-разделяемый* объект» - нонсенс. Объект класса UnsharedConcreteFlyweight не будет являться легковесом, то есть не будет *разделяемым* и соответственно не будет происходить выделения части его *внутреннего* состояния во *внешнее*. Тем не менее GoF включили в имя данного участника слово *Flyweight*, допуская, что не все подклассы класса Flyweight должны быть разделяемыми. Такой подход к именованию данного участника спорный.

FlyweightFactory – **Фабрика легковесов**:
Создает *разделяемые* и *неразделяемые* объекты. Когда клиент запрашивает *разделяемый* объект, фабрика ищет этот объект в «*коллекции легковесов*» и если находит, то возвращает ссылку на него, иначе создает новый объект, сохраняет его в «*коллекции легковесов*» и возвращает ссылку на созданный объект. *Неразделяемые* объекты каждый раз создаются заново.

Отношения между участниками

Отношения между классами

- Класс ConcreteFlyweight связан связью отношения наследования с абстрактным классом Flyweight.
- Класс UnsharedConcreteFlyweight связан связью отношения наследования с абстрактным классом Flyweight.
- Класс FlyweightFactory связан связью отношения агрегации с абстрактным классом Flyweight.

Отношения между объектами

- Состояние, используемое при работе с ConcreteFlyweight разделяется на внутреннее и внешнее. Внутреннее состояние хранится непосредственно в самом объекте ConcreteFlyweight. Внешнее состояние хранится в клиенте (Client) и передается легковесу в качестве аргументов методов.
- Клиенты не создают экземпляры ConcreteFlyweight напрямую через вызов конструктора, а получают их от объекта FlyweightFactory. Такой подход позволяет гарантированно получить разделяемый объект.
- Клиентам рекомендуется создавать экземпляры UnsharedConcreteFlyweight при помощи объекта FlyweightFactory, так как в ходе эволюции программы может потребоваться преобразовать неразделяемые объекты в разделяемые и если неразделяемые объекты создавались напрямую через вызов конструктора, то придется вносить изменения в клиентский код.

Применение

Когда не хватает оперативной памяти для поддержки всех нужных объектов.

Эффективность паттерна **Легковес** во многом зависит от того, как и где он используется.

Применять этот паттерн следует, когда выполнены все перечисленные условия:

- в приложении используется большое число объектов;
- из-за этого высоки расходы оперативной памяти;
- большую часть состояния объектов можно вынести за пределы их классов;
- большие группы объектов можно заменить относительно небольшим количеством разделяемых объектов, поскольку внешнее состояние вынесено.

Шаги реализации

1. **Разделить поля класса, который станет легковесом, на две части**:
   - внутреннее состояние: значения этих полей одинаковы для большого числа объектов;
   - внешнее состояние (контекст): значения полей уникальны для каждого объекта.
2. **Оставить поля внутреннего состояния в классе**, убедившись, что их значения неизменяемы. Эти поля должны инициализироваться только через конструктор.
3. **Преобразовать поля внешнего состояния в параметры методов**, где эти поля использовались. Затем удалить поля из класса.
4. **Создать фабрику, которая будет кешировать и повторно отдавать** уже созданные объекты. Клиент должен запрашивать из этой фабрики легковеса с определённым внутренним состоянием, а не создавать его напрямую.
5. **Клиент должен хранить или вычислять значения внешнего состояния** (контекст) и передавать его в методы объекта легковеса.

Преимущества и недостатки

+ Экономит оперативную память.

- Расходует процессорное время на поиск/вычисление контекста.

- Усложняет код программы из-за введения множества дополнительных классов.